

Table of Contents

#	Section	Contents
1	Overview & Research Question	Problem definition, quantum threat context
2	Cryptographic Background	RSA, Shor's Algorithm, post-quantum standards
3	Prerequisites	Tools, libraries, environment setup
4	Project Structure	Folder layout and file organization
5	RSA Benchmarking	Key generation, encryption, decryption timing
6	Growth Coefficient Analysis	Superlinear scaling of RSA decryption cost
7	Kyber (ML-KEM) Benchmarking	Post-quantum key encapsulation performance
8	Results & Comparison	Side-by-side analysis, charts, key findings
9	Git & GitHub	Version control workflow
10	Glossary	Technical terms explained

1. Overview & Research Question

This project investigates the real-world performance implications of transitioning from classical RSA cryptography to post-quantum ML-KEM (Kyber) — the algorithm standardized by NIST in 2024 as the primary post-quantum key encapsulation mechanism.

1.1 Research Question

"What is the performance cost of transitioning from classical RSA to post-quantum ML-KEM (Kyber) cryptography, and how vulnerable are today's systems against a quantum-capable adversary?"

1.2 Motivation

RSA is the foundation of HTTPS, digital signatures, and secure communications worldwide. Its security relies on the computational hardness of integer factorization — a problem that classical computers cannot solve efficiently for large keys. However, Shor's Algorithm running on a sufficiently powerful quantum computer solves this in polynomial time, rendering RSA cryptographically broken.

The consensus in the cryptographic community is that a cryptographically relevant quantum computer (CRQC) will exist between 2030 and 2040. The immediate threat is 'Harvest Now, Decrypt Later' — adversaries are already collecting encrypted traffic today, intending to decrypt it once quantum hardware matures.

✓ NIST standardized ML-KEM (formerly Kyber) in August 2024 as FIPS 203 — the first post-quantum cryptography standard for key encapsulation.

2. Cryptographic Background

2.1 RSA — How It Works

RSA (Rivest-Shamir-Adleman, 1977) is an asymmetric cryptosystem based on the mathematical difficulty of factoring the product of two large prime numbers. A public key is used to encrypt data; only the corresponding private key can decrypt it.

Security scales with key size: RSA-2048 is currently considered secure for classical computers, requiring billions of years to factor with today's best algorithms. RSA-4096 provides a larger margin but at significant computational cost.

2.2 Shor's Algorithm — The Quantum Threat

In 1994, Peter Shor demonstrated that a quantum computer could factor large integers in polynomial time using quantum superposition and interference. This means a sufficiently powerful quantum computer could break RSA-2048 in hours or days — not billions of years.

The implication: increasing RSA key sizes is not a sustainable defense. This project quantifies exactly how unsustainable it is through the **growth coefficient** — a metric we derive from empirical benchmarks.

2.3 ML-KEM (Kyber) — Post-Quantum Alternative

Kyber is a Key Encapsulation Mechanism (KEM) based on the Module Learning With Errors (M-LWE) problem — believed to be hard for both classical and quantum computers. It does not encrypt data directly; instead it establishes a shared secret key that can then be used with symmetric encryption (AES).

Three security levels mirror NIST AES equivalents: Kyber512 (AES-128), Kyber768 (AES-192), Kyber1024 (AES-256).

3. Prerequisites

3.1 Required Tools

Tool	Version	Purpose	Install
Python	3.14+	Primary programming language	python.org/downloads
Git	2.54+	Version control	git-scm.com/downloads
VS Code	Latest	Code editor	code.visualstudio.com
Git Bash	Bundled	Unix-compatible terminal	Included with Git
Jupyter	via pip	Interactive notebook environment	<code>pip install jupyter</code>

✓ Git Bash provides a Unix shell on Windows — standard POSIX commands (`mkdir`, `chmod`, `echo`, `rm`) mirror the Linux environment used in most research computing and cloud infrastructure.

3.2 Python Libraries

```
pip install cryptography pycryptodome matplotlib pandas numpy jupyter kyber-py
```

Library	Purpose
cryptography	RSA key generation, encryption, decryption with OAEP padding
pycryptodome	Additional cryptographic primitives and low-level operations
kyber-py	Pure Python implementation of Kyber/ML-KEM post-quantum KEM
pandas	DataFrame manipulation for benchmarking results
matplotlib	Chart generation for performance comparison
numpy	Numerical operations and array handling
jupyter	Interactive notebook environment for reproducible research

3.3 PATH Configuration in Git Bash

```
export PATH="$PATH:/c/Users/YOUR_USER/AppData/Roaming/Python/Python314/Scripts"
```

- Required each session — add to `~/.bashrc` to make permanent
- Ensures `pip`-installed executables (`jupyter`, etc.) are found by the shell

4. Project Structure

```
post-quantum-risk-analysis/  
■■■ data/                # Reserved for external datasets (empty – data is generated)  
■■■ notebooks/           # Jupyter analysis notebooks  
■   ■■■ analysis.ipynb  
■■■ src/                 # Modular Python source code (future)  
■■■ docs/               # Generated outputs  
■   ■■■ analysis.html    # Exported notebook with full outputs  
■   ■■■ performance_comparison.png  
■■■ requirements.txt      # Python dependencies  
■■■ .gitignore           # Files excluded from Git  
■■■ LICENSE              # MIT License  
■■■ README.md            # Project documentation
```

■ *data/* — empty here because all data is generated programmatically by the benchmarks

■ *docs/* — contains the HTML export and PNG chart for easy viewing on GitHub

■ *requirements.txt* — generated with *pip freeze*; enables one-command environment reproduction

✓ Reproduce the environment on any machine with: `pip install -r requirements.txt`

5. RSA Benchmarking

5.1 Imports

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend
from kyber_py.kyber import Kyber512, Kyber768, Kyber1024
```

■ *time* — standard library for high-resolution timing

■ *cryptography.hazmat* — low-level cryptographic primitives ('*hazmat*' = hazardous materials; use with care)

■ *default_backend()* — selects the best available cryptographic backend for the platform

5.2 Key Generation Benchmark

```
key_sizes = [1024, 2048, 4096]
rsa_results = []

for size in key_sizes:
    start = time.time()
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=size,
        backend=default_backend()
    )
    elapsed = time.time() - start
    public_key = private_key.public_key()
    rsa_results.append({'key_size': size, 'keygen_time': elapsed})
    print(f"RSA-{size}: {elapsed:.4f}s")
```

■ *public_exponent=65537* — industry standard; $65537 = 2^{16} + 1$, chosen for efficiency and security

■ *key_size* minimum is 1024 bits — the cryptography library enforces this as a security floor

■ **RSA-512 and below are cryptographically broken and rejected by modern libraries.**

5.3 Encryption & Decryption Benchmark

```
message = b"Sensitive data requiring quantum-resistant protection."

# For each key size:
ciphertext = public_key.encrypt(
    message,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

■ *OAEP* — Optimal Asymmetric Encryption Padding; modern secure padding scheme for RSA

■ *MGF1* — Mask Generation Function; used internally by OAEP

- *SHA256 — hash function used within the padding scheme*
- *Decryption uses the same padding in reverse with the private key*

5.4 Results

Key Size	Keygen Time	Encrypt Time	Decrypt Time
RSA-1024	0.0714s	0.001493s	0.000633s
RSA-2048	0.0358s	0.000158s	0.000998s
RSA-4096	0.4347s	0.000141s	0.004026s

Notable observation: encryption time is nearly constant across key sizes because it uses the public key (simple modular exponentiation). Decryption time grows significantly because it uses the private key (computationally expensive operation).

6. Growth Coefficient Analysis

A key contribution of this project is quantifying *how much* RSA decryption cost grows relative to key size increases. We define the **Growth Coefficient** as:

```
growth_coefficient = time_ratio / key_ratio

# Where:
# time_ratio = decrypt_time[i] / decrypt_time[i-1]
# key_ratio  = key_size[i]   / key_size[i-1]
```

- A coefficient of 1.0 means time grows proportionally to key size (*linear*)
- A coefficient above 1.0 means time grows faster than key size (*superlinear*)

6.1 Results

Transition	Key Ratio	Time Ratio	Growth Coefficient	Interpretation
RSA-1024 → RSA-2048	2x	1.58x	0.79	Sub-linear (cache effects)
RSA-2048 → RSA-4096	2x	4.03x	2.02	Superlinear growth

The RSA-2048 → RSA-4096 transition shows a growth coefficient of 2.02 — doubling the key size more than quadruples decryption time. This confirms that key-size scaling is computationally unsustainable as a long-term defense strategy against quantum adversaries.

✓ This growth coefficient is an original analytical contribution of this project — a quantitative metric for evaluating the sustainability of RSA key scaling.

7. Kyber (ML-KEM) Benchmarking

7.1 Key Encapsulation Mechanism

Unlike RSA which directly encrypts data, Kyber is a KEM — it establishes a shared secret key between two parties. That shared key is then used with a symmetric cipher like AES. This is the standard hybrid encryption approach in modern protocols.

```
from kyber_py.kyber import Kyber512, Kyber768, Kyber1024

# Key generation
public_key, private_key = Kyber512.keygen()

# Encapsulation (sender)
shared_key, ciphertext = Kyber512.encaps(public_key)

# Decapsulation (receiver)
decoded_key = Kyber512.decaps(private_key, ciphertext)
```

- `keygen()` — generates a public/private key pair
- `encaps()` — sender generates a shared secret and ciphertext using recipient's public key
- `decaps()` — recipient recovers the shared secret using their private key and the ciphertext
- `shared_key == decoded_key` — both parties now have the same secret for symmetric encryption

■ Note: kyber-py is a pure Python implementation for research and benchmarking. Production use should use NIST-validated C implementations (liboqs, BoringSSL).

7.2 Results

Variant	Security Level	Public Key	Ciphertext	Keygen	Encaps	Decaps
Kyber512	AES-128	800B	768B	0.00580s	0.00543s	0.00608s
Kyber768	AES-192	1184B	1088B	0.00828s	0.00585s	0.01553s
Kyber1024	AES-256	1568B	1568B	0.01002s	0.00467s	0.00585s

8. Results & Comparison

8.1 Side-by-Side Comparison

Metric	RSA-2048	RSA-4096	Kyber768	Kyber1024
Key Generation	0.0358s	0.4347s	0.00828s	0.01002s
Encrypt/Encaps	0.000158s	0.000141s	0.00585s	0.00467s
Decrypt/Decaps	0.000998s	0.004026s	0.01553s	0.00585s
Public Key Size	256B	512B	1184B	1568B
Quantum Safe	NO	NO	YES	YES

8.2 Key Findings

- 1. RSA key scaling is superlinear:** doubling the key size from 2048 to 4096 increases decryption time by 4x, with a growth coefficient of 2.02. Larger keys buy time, not security against quantum adversaries.
- 2. Kyber is computationally viable:** all three Kyber variants complete key generation, encapsulation, and decapsulation in under 20ms. The performance overhead compared to RSA decryption is marginal and acceptable.
- 3. Key sizes are larger in Kyber:** Kyber1024 public keys are 1568 bytes vs. 512 bytes for RSA-4096. This is a trade-off — larger keys for quantum resistance is a one-time infrastructure cost, not an ongoing computational one.
- 4. Migration is urgent:** 'Harvest Now, Decrypt Later' attacks mean that data encrypted today with RSA may be compromised once quantum hardware matures. Systems handling long-lived sensitive data should prioritize post-quantum migration now.

✓ Conclusion: post-quantum migration via ML-KEM (Kyber) is computationally viable today. The performance overhead is acceptable; the security gain is necessary.

9. Git & GitHub

9.1 Repository Initialization

```
git init
git remote add origin https://github.com/USER/post-quantum-risk-analysis.git
git pull origin main
```

- *git init* — initializes an empty Git repository in the current directory
- *git remote add origin* — links the local repo to the remote GitHub repository
- *git pull origin main* — fetches and merges the remote main branch locally

9.2 Standard Workflow

```
git add .
git commit -m "feat: description"
git push origin main
```

- *Always use git push origin main explicitly* — avoids ambiguity with multiple remotes

9.3 Exporting the Notebook

```
jupyter nbconvert --to html notebooks/analysis.ipynb --output ../docs/analysis.html
```

- *nbconvert* — converts *.ipynb* to *HTML* with all outputs rendered
- *../docs/* — saves to the docs folder at the project root, not inside notebooks/

■ Always use *../docs/* and not *docs/* when running *nbconvert* from inside the *notebooks/* directory. Using *docs/* creates a nested *docs/docs/* folder.

10. Glossary

Term	Definition
RSA	Rivest-Shamir-Adleman. Asymmetric cryptosystem based on integer factorization.
Asymmetric crypto	Uses a key pair: public key to encrypt, private key to decrypt.
Public exponent	65537 is standard for RSA. Chosen for fast modular exponentiation.
OAEP	Optimal Asymmetric Encryption Padding. Secure padding scheme for RSA.
SHA-256	Secure Hash Algorithm 256-bit. Used within OAEP padding.
Shor's Algorithm	Quantum algorithm that factors integers in polynomial time, breaking RSA.
CRQC	Cryptographically Relevant Quantum Computer. Capable of running Shor's at scale.
Post-quantum	Cryptographic algorithms believed secure against both classical and quantum computers.
NIST	National Institute of Standards and Technology. Standardized ML-KEM in 2024.
ML-KEM	Module Lattice Key Encapsulation Mechanism. NIST standard name for Kyber (FIPS 203).
KEM	Key Encapsulation Mechanism. Establishes a shared secret; does not encrypt data directly.
M-LWE	Module Learning With Errors. The hard mathematical problem underlying Kyber.
Encapsulation	Sender generates shared secret + ciphertext using recipient's public key.
Decapsulation	Recipient recovers the shared secret using their private key and the ciphertext.
Growth Coefficient	Ratio of time growth to key size growth. Values above 1 indicate superlinear scaling.
Harvest Now	Attack strategy: collect encrypted data today, decrypt later with quantum hardware.
Hybrid encryption	Use KEM to exchange a key, then encrypt data with fast symmetric cipher (AES).
FIPS 203	Federal standard number for ML-KEM, published by NIST in August 2024.
pip freeze	Lists all installed Python packages with exact versions. Used to create requirements.txt.
nbconvert	Jupyter tool for exporting notebooks to HTML, PDF, script, and other formats.